

CHRONO SUPPORT FOR HUMAN-IN-THE-LOOP SIMULATION

Kyle Sha
Simulation-Based Engineering Lab

What counts as human-in-the-loop

the definition this tutorial works to, and the three things it demands of a simulator

The definition this tutorial works to

- A simulation is human-in-the-loop when all three of these hold:
 - 1. a person's action changes the state of a RUNNING simulation,
 - 2. the simulation's response changes what that person does next, and
 - 3. the cycle closes fast enough that they can keep reacting.
- Each clause is load-bearing:
 - drop 1 and you have a visualisation - you are watching, not acting
 - drop 2 and the person is driving blind - open loop, not a loop
 - drop 3 and you have a batch job with a joystick attached

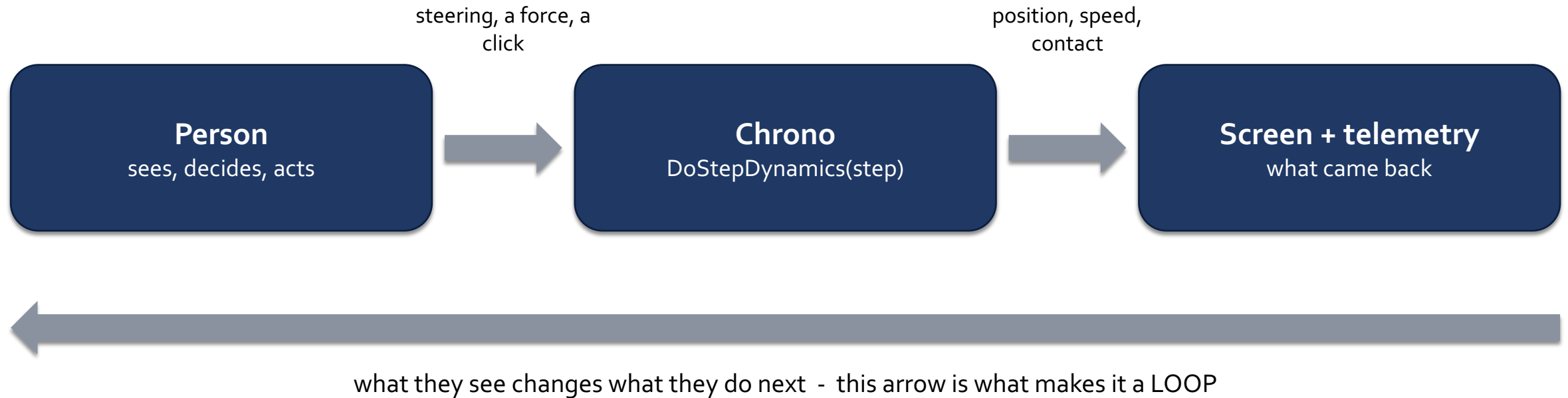
Nothing in that definition mentions vehicles. That is the whole point of the tutorial: a car, a crane, a gearbox and a quadruped's leg all satisfy it, and they satisfy it the same way.

Why a person at all

- A gantry crane. Four bodies, two speed motors, one distance constraint. No vehicle, no terrain, no tires.
- The payload hangs free and NOTHING damps the swing but the operator.
 - accelerate hard and the load swings
 - it keeps swinging until someone drives the trolley back under it
- `status()` reports the swing angle, so the console scores how badly you are doing without your having to watch the window.

No controller in this tutorial can land that load, which is the cleanest answer to 'why not just automate it'. It also settles the generality question early: if the pattern reaches a crane, it is not a Chrono::Vehicle feature.

The loop, and the clock around it



and every lap has to fit inside one wall-clock step

Break the first arrow and it is a movie. Break the return arrow and it is a recording session. Break the clock and the person cannot stay in the loop at all, which is why the clock comes first in this tutorial.

Not everything interactive is in the loop

- IN the loop - a person's input is a term in the dynamics, every step
 - driving the HMMWV; dragging a robot's limb while its controller resists
- ON the loop - a person configures and triggers, then watches it play out
 - the push rig: you choose where and how hard, the impulse itself is scripted
- Neither - interactive, but nothing is reacting
 - the top-down placement tool: kinematic, no dynamics, a scene-authoring tool

Worth saying out loud, because 'interactive' and 'in the loop' get used interchangeably and only one of them has a clock in it. All three are useful. Only the first two are what the phrase means.

Requirement 3 comes first: the clock

it decides whether the other two requirements are worth anything

What real time means in Chrono

- Chrono integrates as fast as it can. It has no notion of wall-clock time.
- $RTF = \text{wall time} / \text{simulated time}$
 - $RTF < 1$ - faster than real time. You have slack, and can sleep.
 - $RTF = 1$ - exactly real time.
 - $RTF > 1$ - slower than real time. No amount of sleeping fixes this.
- So holding the clock is two separate problems:
 - do not run FAST - sleep off the slack (easy, one call)
 - do not run SLOW - make the step cheap enough to have slack (the real work)

Unpaced, a box being dragged by hand travelled 88.7 m in one wall-clock second instead of 1.8 m. The physics was right both times. Only one of them was usable by a person.

Holding wall-clock time: the pacing surface



```
// chrono/core/ChRealtimeStep.h
class ChRealtimeStepTimer : public ChTimer {
    void Spin(double step);    // sleep off whatever is left of `step`
};

// chrono_vehicle/ChVehicle.h -- same policy, applied inside Advance()
void  ChVehicle::EnableRealtime(bool val);
double ChVehicle::GetRTF() const;    // wall / simulated, per step

// The pattern, whatever is being simulated:
//   while running:
//       inputs = read_human()        # never blocking
//       sys.DoStepDynamics(step)
//       if time to draw: render()
//       timer.Spin(step)             # give the wall clock its due

# ---- and how this tutorial calls it (tutorial_HIL_driver.py, lines 812 to 816) -----
# PART 1: spin in place for real time to catch up
if realtime_mode == "per_step":
    rt_timer.Spin(step_size)
elif realtime_mode == "cumulative":
    cum_timer.spin(system.GetChTime())
```

Two calls. Everything else about real time is arithmetic around them.

Per-step timers never recover time lost on a slow step. If you need the sim to catch up after a stall, pace against TOTAL elapsed time instead, which is the third mode in this tutorial.

Show Time

- `REALTIME = "none"`: watch the drift column run away
- `REALTIME = "vehicle"`: drift pinned near zero, RTF at 1
- Same physics in both runs. The only difference is whether the process sleeps.

The drift column is the honest one. RTF can look fine on average while the sim has already lost a second of wall clock.

You can only sleep if you have slack

- Holding the clock is two problems, and only one of them is a call:
 - do not run FAST - sleep off the slack
 - do not run SLOW - make the step cheap enough to HAVE slack
- The budget is fixed: one step of wall clock, minus what you spend drawing, minus what you leave as margin for a bad step.
- What follows is the set of levers Chrono gives you for the second problem, in rough order of how much they usually buy.

Reach for them in this order. People tend to start at the solver, which is near the bottom of the list and the easiest place to spend an afternoon for nothing.

The levers, in rough order of payoff

- 1. Step size. Everything scales with steps per second. The largest step that stays stable is the single biggest lever there is.
- 2. Collision geometry. Primitives beat convex hulls beat triangle meshes, by a lot. Most imported assets arrive as the slowest option.
- 3. Which pairs are tested at all. Families and masks remove whole classes of pair from the broadphase, including self-collision.
- 4. Active domains. Let the expensive physics happen only where it matters: deformable terrain near the wheels, fluid near the hull.
- 5. Solver type and iteration cap. Iterations buy convergence and cost time, and you pay out of the same step you wanted to sleep in.
- 6. Threads, sleeping bodies, and model reduction for elastic subassemblies.
- 7. Render less often than you step. Drawing is not free, and the eye does not need 500 Hz.

None of this is specific to having a human in the loop. It becomes urgent when there is one, because a batch job that runs at 0.6x real time is merely slow, and an interactive one is broken.

The levers, by name

```
// the step, and how it is integrated
sys.DoStepDynamics(step);
sys.SetTimestepperType(ChTimestepper::Type::...);

// the solver: type, and how long it is allowed to work
sys.SetSolverType(ChSolver::Type::BARZILAIBORWEIN);
sys.GetSolver()->AsIterative()->SetMaxIterations(n);

// what the collision system is asked to do
model->SetFamily(i); model->SetFamilyMask(mask); // skip whole pairs
ChCollisionModel::SetDefaultSuggestedEnvelope(e); // contact envelope
ChCollisionModel::SetDefaultSuggestedMargin(m);

// where the expensive physics is allowed to happen
SCMTerrain::AddActiveDomain(body, ...); // deform near the wheels only
ChFsiSystemSPH::SetActiveDomain(box); // fluid where it matters only

// spend less on what is not moving, or not detailed
sys.SetNumThreads(n_chrono, n_collision, n_eigen); // three pools
sys.SetSleepingAllowed(true); // bodies at rest drop out
ChModalAssembly::DoModalReduction(...); // internal DOFs -> a few modes
```

Every one of these is a real-time decision before it is a fidelity decision, and most of them are one line.

Two that do not look like performance problems

- Under-resolved friction reads as a frictionless floor. A solver that runs out of iterations before the tangential constraints converge does not report an error: the thing just slides, and you go hunting through friction coefficients instead of iteration counts.
- Imported collision geometry is authored for a different question. A planner's mesh answers 'do these two overlap', once. A dynamics loop asks every step, for every pair, and the file does not know which job it is doing.
- Switching one quadruped's URDF triangle meshes to convex hulls, and masking self-collision, took it from 427 contacts standing still to 6, and from slower than real time to roughly fifteen times faster.

Both of these cost days somewhere. They are worth naming out loud because neither presents as a speed problem, and the instinct in both cases sends you somewhere useless.

Requirement 1: a path into the state

how a person's action actually reaches the equations

Three ways a person's input enters a Chrono simulation

As driver inputs

ChDriver / ChInteractiveDriver
steering, throttle, braking

the vehicle demos (Parts 2-4)

IN the loop

As a force or constraint

AccumulateForce, ChLinkTSDA
on any body, any scene

the drag and push demos (Parts 9-10)

IN the loop

As a pose you set

SetPos / SetRot
kinematic placement

the placement tool

not in the loop

The middle column is the general one. A vehicle has a driver class because somebody wrote one; every other plant in this tutorial takes its human input as a force, which is why the same pattern reaches a crane, a gearbox and a quadruped.

The right-hand column is honest about itself: nothing reacts to a pose you set, so nothing closes the loop.

Feeding driver input: the ChDriver surface



```
// chrono_vehicle/ChDriver.h -- what EVERY driver exposes, human or not
class ChDriver {
    void SetSteering(double s);    // -1 .. +1
    void SetThrottle(double t);    //  0 .. 1
    void SetBraking(double b);    //  0 .. 1
    void SetClutch(double c);     //  0 .. 1
    DriverInputs GetInputs() const;
    virtual void Synchronize(double time); // read the world
    virtual void Advance(double step);     // advance own state
};

// Two ways in, and the vehicle cannot tell them apart:
//   subclass ChDriver and write the setters from your own device, or
//   take ChInteractiveDriver and let it read a keyboard or a joystick.
// Either way the vehicle only ever sees a DriverInputs struct.

# ---- and how this tutorial calls it (tutorial_HIL_driver.py, lines 583 to 585) -----
if KEYBOARD_MODE == "held":
    if hasattr(veh.ChInteractiveDriver, "KeyboardMode_HELD"):
        driver.SetKeyboardMode(veh.ChInteractiveDriver.KeyboardMode_HELD)
```

Query the driver BEFORE Synchronize: the human's numbers have to be in hand before the modules read each other for this step.

KeyboardMode: CUMULATIVE nudges an input and leaves it there; HELD follows the keys currently down, like a driving game. HELD was added to Chrono for this tutorial and ships in build 1187.

Input devices Chrono already speaks

- ChInteractiveDriver covers two CLASSES of device, not two devices:
 - InputMode_KEYBOARD - with KeyboardMode_HELD or _CUMULATIVE
 - InputMode_JOYSTICK - anything the windowing layer enumerates as one
- A joystick is mapped by a JSON file, not by code: SetJoystickConfigFile(path)
 - mapped axes: steering, throttle, brake, CLUTCH
 - mapped buttons: shift up/down, reverse, gears 1-9, manual-gearbox toggle, plus one user callback
- Four configs ship in data/vehicle/joystick/: Default, Logitech RumblePad 2, Xbox One, and Wheel+Pedals+Shifters
- Each control names its own DEVICE, so a wheel, a pedal box and an H-shifter enumerating as three separate USB devices all map at once

So a G923-class rig is a config file, not a port. The mapping and the semantics are Chrono's and cross-platform; enumerating the device is delegated to the windowing layer, so which platforms see your wheel follows that layer, and I have not tested it here.

All of it is exposed to PyChrono.

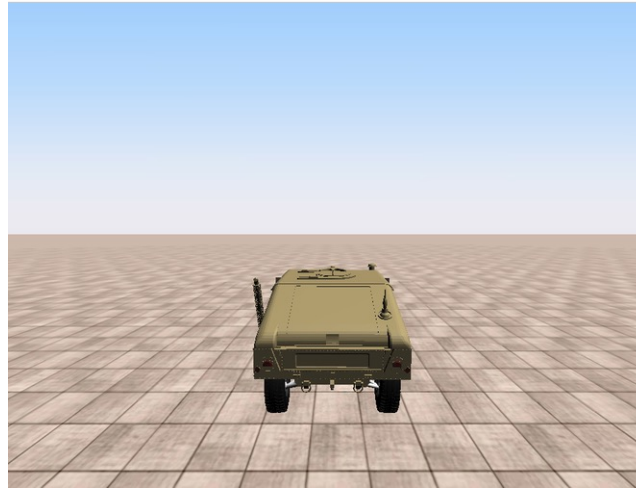
And for a device Chrono does not know about

- Subclass ChDriver, read whatever you like, and obey three rules:
 - 1. NEVER BLOCK. A simulation that waits on a human is not real-time. Late input means hold the last value, not stall the loop.
 - 2. SAMPLE ONCE PER STEP, THEN HOLD. Read at the top of the step and reuse that value for the whole step, or the physics sees inputs change for reasons the operator can neither perceive nor reproduce.
 - 3. SEND SOMETHING BACK. With no return path the operator is driving open loop, which is not human-in-the-loop by our definition.
- Obey those and the transport does not matter: a socket, a serial port, shared memory, a ROS topic, another simulator.

This tutorial ships one of these, an operator console on UDP, so a second machine can drive the simulation and watch telemetry come back.

Show Time

- `INPUT_SOURCE = "keyboard", KEYBOARD_MODE = "held"`
- W/A/S/D drive. The arrow keys are the chase camera, not the car.
- Try "cumulative" and feel the difference a key-state API makes.



This is the whole definition satisfied in one window: input reaches the state, the state comes back on screen, and the timer holds the pace.

Reaching into the scene with a mouse

what Chrono provides today, the one line it is missing, and what that buys

What Chrono gives you today

- `ChRealtimeStepTimer` - the pacing, exposed to Python
- `ChInteractiveDriver` - keyboard and joystick input, with `KeyboardMode::HELD`
- `ChCollisionSystem::RayHit` - click-to-pick, exposed to Python
- `AccumulateForce`, `ChLinkTSDA` - a human's pull as a term in the dynamics
- `ChVisualSystemIrrlicht::AddUserEventReceiver(IEventReceiver*)` - the socket for any input device, and it IS bound in Python
- What is NOT there: any built-in mouse pick-and-drag. No module in the tree implements one.

That last line is the gap this tutorial walks into. Everything needed to build one is already exposed; what is missing is the ability to receive the mouse event in the first place.

The gap, and the one line that closes it (native_pick.py lines 10 to 26)



```
# chrono_swig/interface/irrlicht/ChModuleIrrlicht.i
+%feature("director") irr::IEventReceiver;
#include "IEventReceiver.h"

# The module ALREADY compiles with directors enabled:
# %module(directors="1", threads="1") irrlicht
# and AddUserEventReceiver is ALREADY bound. Without the
# director, though, Python gets an abstract class:
# >>> ci.IEventReceiver()
# AttributeError: No constructor defined - class is abstract
# so there is a socket, and nothing you can plug into it.

class MouseReceiver(irr.IEventReceiver):
    """No Quartz, no CGWindowList, no NSEvent. No macOS. Just Irrlicht."""

    def __init__(self):
        super().__init__()
        self.x = self.y = 0
        self.down = False
        self.pressed = False          # edge, consumed by the loop
        self.keys = set()

    def OnEvent(self, ev):
        if ev.EventType == irr.EET_MOUSE_INPUT_EVENT:
            self.x, self.y = ev.MouseInput.X, ev.MouseInput.Y
            if ev.MouseInput.Event == irr.EMIE_LMOUSE_PRESSED_DOWN:
                self.down = self.pressed = True
            elif ev.MouseInput.Event == irr.EMIE_LMOUSE_LEFT_UP:
                self.down = False
```

Without it, reading the mouse means going around Chrono to the operating system: 135 lines, macOS only, and it breaks when another window takes focus.

With it, the same capability is 56 lines of ordinary PyChrono. Upstream PR to follow this talk.

Turning a click into a body: the ray-cast surface

```
// chrono/collision/ChCollisionSystem.h
struct ChRayhitResult {
    bool hit;
    ChVector3d abs_hitPoint;    // where, in world coordinates
    ChVector3d abs_hitNormal;   // the surface normal there
    double dist_factor;        // 0 .. 1 along the segment
    ChCollisionModel* hitModel; // -> CastToChBody() for the body
};

virtual bool RayHit(const ChVector3d& from,
                   const ChVector3d& to,
                   ChRayhitResult& result) const;

// A ray sees COLLISION geometry, never visual geometry. A link you
// can see but whose collision is off is invisible to every click.
// That one sentence accounted for every 'it will not pick' bug here.

# ---- and how this tutorial calls it (hil_manipulate.py, lines 338 to 349) -----
def pick_along_ray(system, start, end):
    """The primitive a mouse click would use. Returns (body, world_point) or None."""
    res = chrono.ChRayhitResult()
    system.GetCollisionSystem().RayHit(start, end, res)
    if not res.hit:
        return None
    body = chrono.CastToChBody(res.hitModel.GetContactable())
    if body is None:
        return None
    p, nrm = res.abs_hitPoint, res.abs_hitNormal
    return (body, chrono.ChVector3d(p.x, p.y, p.z),
           chrono.ChVector3d(nrm.x, nrm.y, nrm.z))
```

Exposed in PyChrono already, and exact: the same primitive a native mouse handler would call. Screen pixel to ray is your arithmetic; ray to body is Chrono's.

Turning a pull into physics: force, or constraint

```
// A human pulling on something is one of exactly two things.

// 1. A FORCE, when you want a push of known size (chrono/physics/ChBody.h)
unsigned int idx = body->AddAccumulator(); // ONCE per body
body->EmptyAccumulator(idx); // your job, EVERY step
body->AccumulateForce(idx, force, appl_point, local);
body->AccumulateTorque(idx, torque, local);

// 2. A CONSTRAINT, when you want a hand (chrono/physics/ChLinkTSDA.h)
spring->Initialize(handle, body, local, point1, point2);
spring->SetRestLength(0.0);
spring->SetSpringCoefficient(k); // k = m*w^2 scale with the MASS
spring->SetDampingCoefficient(c); // c = 2*m*w critically damped
sys.AddLink(spring);

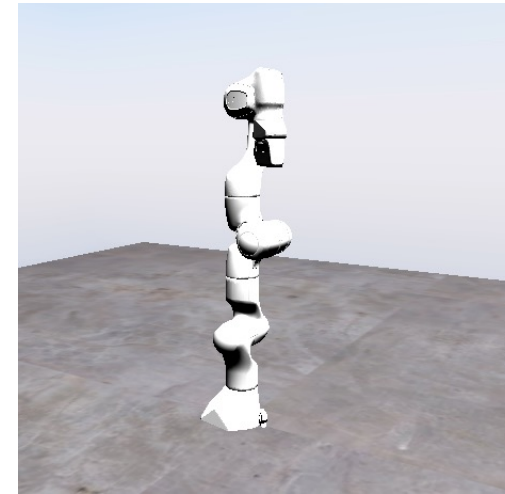
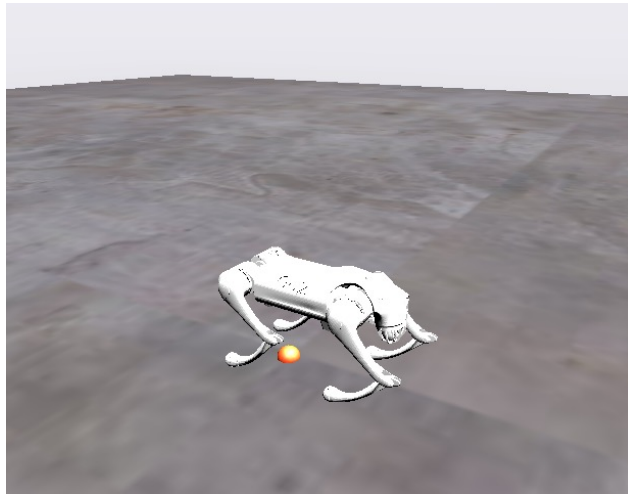
# ---- and how this tutorial calls it (hil_manipulate.py, lines 488 to 502) ----
omega = getattr(self.system, "grab_omega", GRAB_OMEGA)
m = max(body.GetMass(), 1e-3)
k = m * omega * omega
c = 2.0 * m * omega * GRAB_ZETA
# Belt and braces against tunnelling: a grabbed body may not move faster
# than contact detection can keep up with at this step size.
self._had_limit = body.GetLimitSpeed() if hasattr(body, "GetLimitSpeed") else False
body.SetLimitSpeed(True)
body.SetMaxLinVel(GRAB_MAX_SPEED)
self.spring = chrono.ChLinkTSDA()
self.spring.Initialize(self.handle, body, False, point, point)
self.spring.SetRestLength(0.0)
self.spring.SetSpringCoefficient(k)
self.spring.SetDampingCoefficient(c)
self.system.AddLink(self.spring)
```

Chrono never clears an accumulator for you, and `AddAccumulator()` appends, so calling it per push leaks a slot and every stale slot keeps contributing.

Gains must scale with what is being pulled. A stiffness tuned on a 0.154 kg Go2 calf gave a 2.7 kg Panda link 21,870 N/m, which is 24,300 m/s²: through the floor in a single step.

Show Time

- Drag a leg while the stance controller fights back
- Drag a Franka link, hand-guided or fully unactuated
- Yanking a link into the floor: 35.14 m/s before the gains were bounded, 3.25 m/s after



This is the clearest case of the definition on the whole deck: you pull, the controller resists, you feel it resist, and you pull differently.

Taking the human out of the part that must repeat

```
# The pattern, not the API: decide which half of the human's input
# has to be repeatable, and script only that half.
#
#   the person keeps:  WHERE on the body, and WHICH DIRECTION
#   the machine keeps: a constant force F held for a fixed window dt
#
#   impulse  J = F * dt      both printed, both logged
#
# A freehand drag cannot answer 'how hard can I shove it', because
# no two drags are the same: the force depends on how fast a hand
# moved and how long a button was held. There is no number at the end.
# Script the magnitude and you get one you can put in a report.

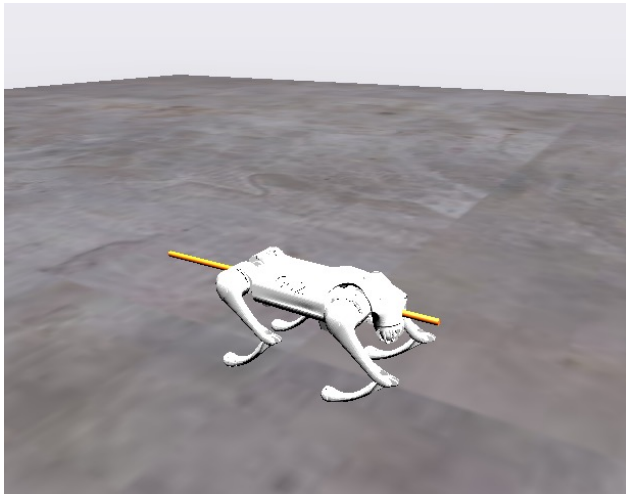
# ---- and how this tutorial calls it (hil_push.py, lines 264 to 283) -----
def fire(self, cfg, t):
    body = cfg.body
    self.active = (body, cfg.force(), chrono.ChVector3d(cfg.local_point),
                  t + cfg.duration)
    self.fired_at = t
    return self._slot(body)

def update(self, t):
    """Call every step, BEFORE DoStepDynamics. Returns True while pushing."""
    if self.active is None:
        return False
    body, force, local, t_end = self.active
    idx = self._slot(body)
    body.EmptyAccumulator(idx)      # Chrono never clears these for you
    if t >= t_end:
        self.active = None
        return False
    # The point travels with the link: a shove on the shoulder stays on the
    # shoulder even as the shoulder is being shoved away.
    body.AccumulateForce(idx, force, body.TransformPointLocalToParent(local),
```

This is the human-ON-the-loop half of the taxonomy: the person sets up the experiment and reads the verdict, and is deliberately not in the inner loop, because being in it would destroy repeatability.

Show Time

- Forward at the base COM: 325 N recovers, 330 N does not
- Sideways: 280 N recovers, 290 N does not
- Same impulse, applied 5 cm higher up the torso: the verdict flips



That last line is the reason a human is still in this one. Where to push is a judgement call, and it changes the answer as much as how hard you push does.

How far this goes

- PART 6 - change gear. A gear is not a fourth float: it belongs to the transmission, and the human sets it through a different API.
- PART 7 - drive somewhere real. SCENE = "mcity" puts the car in the Mcity digital twin.
- PART 8 - control something that is not a car. PLANT = "rover" or "crane", same three floats, no vehicle class involved.
- PARTS 9-10 - reach into the scene: drag a limb, push a robot, place a scene.

The through-line: the loop never changed. What changed was where the human's numbers were injected.

Where this goes next

- Chrono::HIL (github.com/zzhou292/chrono-HIL) - driving rigs, steering wheels, multi-user sessions, the same three floats
- The SWIG director change, as an upstream PR
- A mouse pick-and-drag utility, so that every PyChrono user does not write the same 56 lines
- Policy in the loop: a learned controller on one side, a person perturbing it on the other

The gap this talk found is small and specific, which is the good kind: one line to receive the event, and a utility on top of primitives Chrono already has.

Any questions?

University of Wisconsin - Madison

Thank you.

`github.com/ksha23/chrono-hil-tutorial`